



UNIVERSITÉ
Faculté de Technologie
Département de Génie Civil



CTTP

Contrôle Technique
des Travaux Publics

ALGÉRIE — DÉCEMBRE 1992

Application logicielle pour le dimensionnement du renforcement des chaussées souples *selon le CTTTP Guide des Renforcements (Déc. 1992)*

CTTP RENFORCEMENT

ENSIONNEMENT — DÉFLEXION — MATRICE DE DÉCIS

Application conforme au Guide CTTTP — RN120 PK70–80, Tiaret

Chapitre : Application

RN120 PK70–80 — Wilaya de Tiaret, Algérie

Rédigé par : Étudiant(e) en Master 2 — Génie Civil

Encadré par : Direction des Études Techniques (DET), Tiaret

Organisme : CTTP — Contrôle Technique des Travaux Publics

Année universitaire 2025–2026

Table des matières

1	Application : conception, mise en œuvre et validation de l'outil CTPP	
	Renforcement	4
1.1	Introduction	4
1.2	Présentation générale de l'application	5
1.2.1	Contexte opérationnel	5
1.2.2	Cahier des charges fonctionnel	5
1.2.3	Périmètre normatif	6
1.3	Architecture du système	7
1.3.1	Vue d'ensemble	7
1.3.2	Pile technologique	8
1.3.3	Structure du projet	9
1.4	Module de calcul CTPP	10
1.4.1	Modèle de données	10
1.4.2	Calcul du trafic	11
1.4.3	Correction de la déflexion	11
1.4.4	Matrice de décision	12
1.4.5	Catalogue matériaux	13
1.4.6	Traçabilité réglementaire	14
1.5	Module d'analyse par intelligence artificielle	14
1.5.1	Architecture du provider d'inférence	14
1.5.2	Provider Gemini	15
1.5.3	Provider ONNX local	15
1.5.4	Provider Python (Keras + YOLOv8)	16
1.6	Interface utilisateur	16
1.6.1	Principes de conception	16
1.6.2	Page de calcul (« Design »)	16
1.6.3	Page d'analyse (« Analysis »)	17
1.6.4	Génération de rapport PDF	18
1.7	Validation fonctionnelle et tests	19
1.7.1	Stratégie de test	19

1.7.2	Cas de test de référence	20
1.7.3	Test de bout en bout documenté	20
1.8	Déploiement multiplateforme	21
1.8.1	Web (Vercel)	21
1.8.2	Application de bureau (Tauri v2)	21
1.8.3	Progressive Web App (PWA)	21
1.8.4	Indicateurs de performance	21
1.9	Limites et perspectives	22
1.9.1	Limites identifiées	22
1.9.2	Perspectives d'évolution	22
1.10	Conclusion	23

Table des figures

1.1	Architecture en couches de l'application CTTP Renforcement : frontal Web, routes d'API et moteurs d'inférence.	7
1.2	Flot de données de l'application : saisie, calcul CTTP, analyse IA, génération du rapport PDF.	8
1.3	Structure du projet (diagramme arborescent) : dossiers principaux — frontal (src/), modèles d'IA (models/), documentation (docs/) et coquille de bureau (src-tauri/).	10
1.4	Schéma de calcul de la déflexion corrigée (diagramme Mermaid) : $d = d_c \times C_s \times C_r \times C_t$ (d'après CTTP p. 33).	12
1.5	Coupe transversale type d'une structure de renforcement « Très Lourd » pour un trafic T3–T5 (d'après CTTP p. 48–55, diagramme Mermaid). . . .	14
1.6	Architecture du provider d'inférence (diagramme Mermaid) : une interface commune (InferenceProvider) encapsule trois moteurs d'IA interchangeables, sélectionnés par la variable d'environnement <code>INFERENCE_BACKEND</code>	15
1.7	Capture d'écran de l'onglet <i>Design</i> : saisie des paramètres et calcul en temps réel de la déflexion corrigée et de la structure de renforcement. . . .	17
1.8	Capture d'écran de l'onglet <i>Analysis</i> : détection automatique des dégradations avec <i>bounding boxes</i> superposées à l'image de chaussée.	18
1.9	Aperçu du rapport PDF généré : paramètres, calcul de déflexion, structure de renforcement et traçabilité réglementaire.	19

Liste des tableaux

1.1	Synthèse des fonctionnalités de l'application	6
1.2	Pile technologique de l'application	9
1.3	Bornes des classes de trafic (cumul de poids lourds > 5 t).	11
1.4	Table de correction thermique C_t (chaussées avec couche bitumineuse $\geq$ 10 cm).	12
1.5	Bornes des zones de déflexion (en 1/100 mm).	12
1.6	Exemples de structures issues des catalogues matériaux.	13
1.7	Cas de test de validation de la matrice de décision.	20

Chapitre 1

Application : conception, mise en œuvre et validation de l’outil CTTP Renforcement

1.1 Introduction

Ce chapitre constitue le volet applicatif du mémoire. Il expose la traduction concrète des spécifications du CTTP – *Guide des Renforcements des Chaussées Souples* de décembre 1992 [1] – en une chaîne logicielle fonctionnelle, utilisable aussi bien en bureau d’études que sur le terrain.

L’objectif principal de l’application est triple :

1. **Automatiser** le calcul réglementaire du renforcement (trafic, déflexion, zone, type et structure) en supprimant les calculs manuels sources d’erreurs ;
2. **Standardiser** la sortie en garantissant la traçabilité complète : chaque résultat cite la page du CTTP dont il est issu ;
3. **Augmenter** l’auscultation visuelle par un module d’analyse d’images basé sur l’intelligence artificielle, capable d’identifier et de localiser les dégradations de surface.

L’application livrée, dénommée CTTP Renforcement, se présente sous la forme d’une *Single Page Application* (SPA) fonctionnant en mode web classique, en *Progressive Web App* (PWA) et en application de bureau native grâce à la coquille Tauri v2. Cette portabilité a été pensée pour répondre aux contraintes opérationnelles : usage en salle d’études (Bureau d’Études Techniques), mais aussi en relevé de terrain sur ordinateur portable.

1.2 Présentation générale de l'application

1.2.1 Contexte opérationnel

Le projet CTTP Renforcement a été développé pour le compte de la Direction des Études Techniques (DET) de Tiaret, dans le cadre de la réhabilitation de la route nationale **RN120 entre les points kilométriques 70 et 80**. Ce tronçon, à fort trafic lourd, présente des dégradations avancées nécessitant un dimensionnement de renforcement rigoureux.

L'application a été conçue pour des ingénieurs civils et des spécialistes chaussées : un public expert qui maîtrise la méthodologie CTTP et qui attend un outil précis, dense en information et traçable. La consultation de l'*anti-référentiel* du produit confirme cette orientation : « le ton est expert-à-expert, pas de paternalisme » (cf. PRODUCT.MD).

1.2.2 Cahier des charges fonctionnel

Le tableau [1.1](#) synthétise les fonctionnalités livrées.

Table 1.1. Synthèse des fonctionnalités de l'application

Fonctionnalité	Description
Saisie des paramètres	Entrée de la classe de trafic (T0–T5), de la surface existante (BB, ES, GNT), de l'UNI, de la déflexion mesurée et du contexte géographique.
Calcul du trafic	Application des formules T_{pl} , T_{ms} et T_c du CTTP p. 19, avec coefficients de répartition par configuration de voies.
Correction de la déflexion	Application successive des coefficients C_s , C_r , C_t et détermination de la zone (Low / Medium / High).
Dimensionnement du renforcement	Recherche dans la matrice 3D (Groupe de trafic $\times$ Acceptabilité $\times$ Zone) du type de renforcement adéquat.
Catalogue matériaux	Sélection des matériaux (GB, GNT), épaisseurs de couche de base, liants et exigences de compactage.
Analyse IA d'images	Détection automatique des dégradations via Gemini (cloud), ONNX (local) ou Keras+YOLOv8 (inférence Python).
Rapport PDF	Génération automatique d'un rapport technique complet avec traçabilité réglementaire.
Mode déconnecté	Fonctionnement <i>priorité hors ligne</i> via <i>processus de service</i> et mémoire cache locale.

1.2.3 Périmètre normatif

L'application implémente fidèlement les prescriptions du CTTP [1] pour les aspects suivants :

- **Classes de trafic** T0–T5 (p. 19 du guide) ;
- **Seuils UNI** en mm/km par type de surface (p. 30–35) ;
- **Formule de déflexion corrigée** $d = d_c \times C_s \times C_r \times C_t$ (p. 33) ;
- **Matrice de décision** (p. 45, tableau 3) pour le choix du type de renforcement ;
- **Catalogue matériaux** pour les structures types (p. 48–55).

Cette fidélité au texte réglementaire constitue la *source de vérité unique* logicielle : toutes les constantes, seuils et matrices sont centralisés dans un module dédié (`cttp-rules.ts`) garantissant qu'une mise à jour du guide ne nécessite qu'une seule modification de fichier source.

1.3 Architecture du système

1.3.1 Vue d'ensemble

L'application suit une architecture *client-serveur* moderne présentée sur la figure 1.1. Le frontal est une application monopage React 19 servie par Next.js 16 (App Router). Elle dialogue avec quatre routes d'API qui matérialisent les frontières fonctionnelles : `/api/design` (moteur CTTP), `/api/predict` et `/api/predict-local` (inférence IA), et `/api/export` (génération PDF).

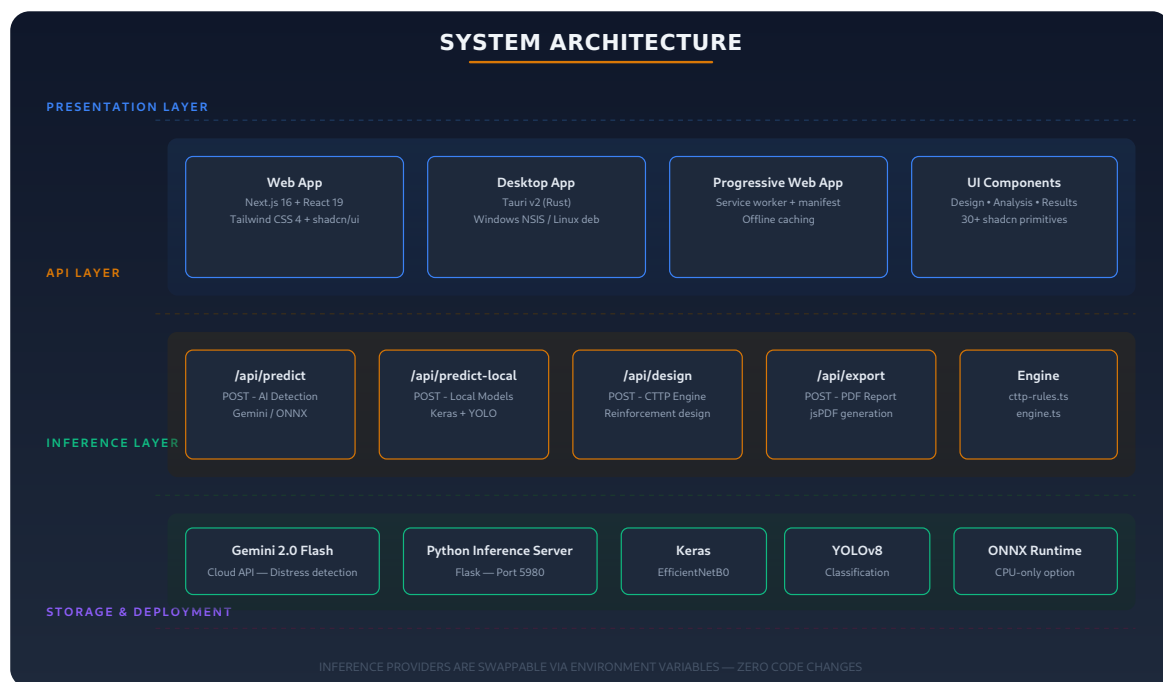


Figure 1.1. Architecture en couches de l'application CTTP Renforcement : frontal Web, routes d'API et moteurs d'inférence.

La figure 1.2 synthétise le flot de données *de bout en bout*, depuis la saisie utilisateur jusqu'à l'export PDF.

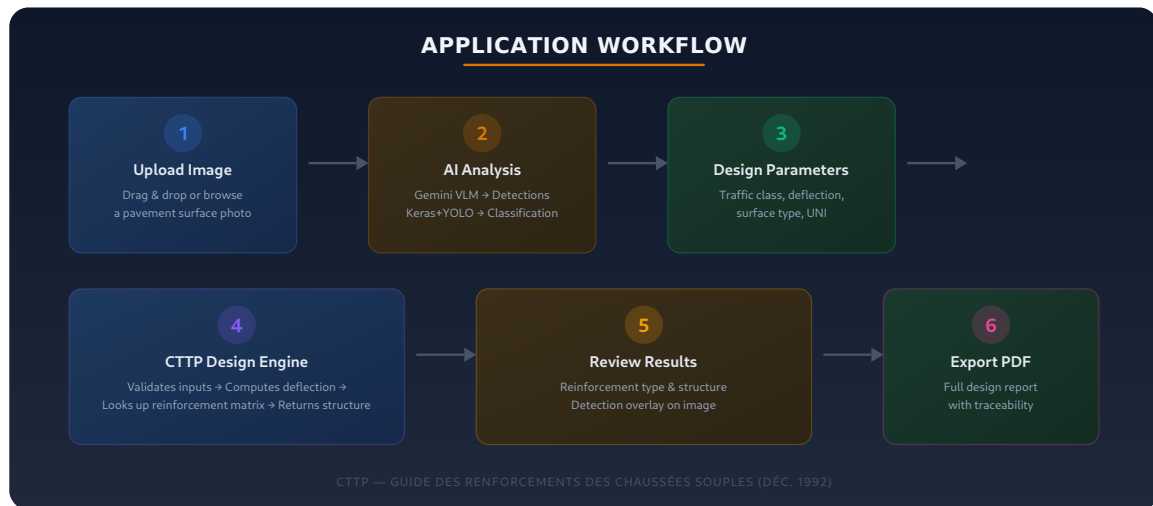


Figure 1.2. Flot de données de l'application : saisie, calcul CTPP, analyse IA, génération du rapport PDF.

1.3.2 Pile technologique

Le tableau 1.2 récapitule les technologies retenues et leur rôle. Le choix d'une *pile* TypeScript de bout en bout garantit l'homogénéité du typage entre le moteur de calcul, les routes d'API et les composants d'interface.

Table 1.2. Pile technologique de l'application

Couche	Technologie			Rôle
Cadriciel frontal	Next.js	16	(App Router)	Application monopage React complète
Interface utilisateur	React	19	+ Tailwind	4 Composants et styles utilitaires
Bibliothèque UI	shadcn/ui	(Radix)		Composants accessibles
Formulaires	react-hook-form		+	Zod Validation et état de formulaire
État	Zustand	+	TanStack	React Query État global et mémoire cache du serveur
i18n	next-intl	Français / Anglais		
Icônes	Lucide	React		Iconographie
Graphiques	Recharts	Visualisation des jauges		
Génération PDF	jsPDF	Rapport côté client		
Calcul CTTP	TypeScript		(moteur dédié)	Règles métier unilatérales
IA – cloud	Gemini	2.0	Flash	VLM de détection distant (Google)
IA – local	ONNX	Runtime	/	YOLOv8-cls Inférence CPU embarquée
IA – local	TensorFlow	/	Keras	(EfficientNetB0) Modèle complémentaire
Serveur d'inférence	Python	Flask		Passerelle HTTP vers Keras/YOLO
Bureau	Tauri	v2 (Rust) Coquille native multi-OS		

1.3.3 Structure du projet

L'arborescence suit la séparation classique *cadriciel* / *logique métier* / *interface utilisateur*. Le moteur CTTP est strictement isolé du frontal et de toute dépendance à React, ce qui le rend théoriquement réutilisable depuis n'importe quel environnement d'exécution Node.

```

1 // Extrait -- cttp-rules.ts
2 export const REINFORCEMENT_MATRIX: MatrixRow[] = [
3   // T0-T1
4   { trafficGroup: "T0-T1", acceptability: "Acceptable",
5     deflectionZone: "Low", reinforcementType: "Léger" },

```

```

6   { trafficGroup: "T0-T1", acceptability: "Non_Acceptable",
7     deflectionZone: "Medium", reinforcementType: "Lourd" },
8   // T2-T3
9   { trafficGroup: "T2-T3", acceptability: "Non_Acceptable",
10    deflectionZone: "High", reinforcementType: "Très Lourd" },
11  // T4-T5 (extrait)
12  { trafficGroup: "T4-T5", acceptability: "Acceptable",
13    deflectionZone: "High", reinforcementType: "Lourd" },
14 ];

```

Listing 1.1. Module central des règles CTTP (cttp-rules.ts).

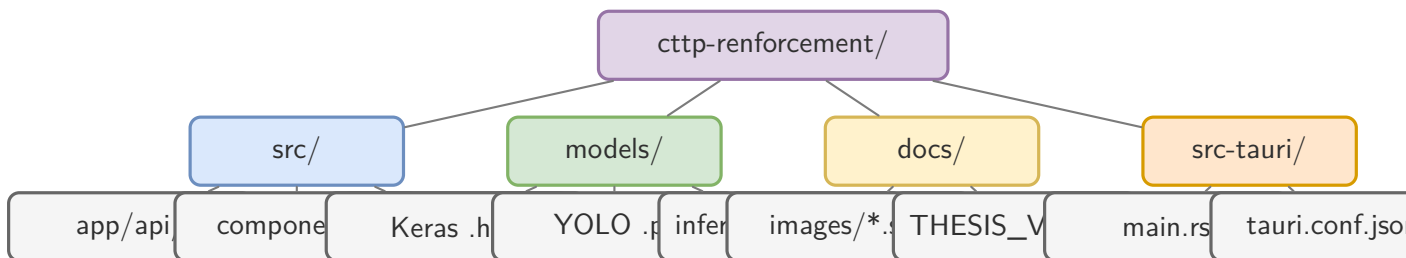


Figure 1.3. Structure du projet (diagramme arborescent) : dossiers principaux — frontal (src/), modèles d’IA (models/), documentation (docs/) et coquille de bureau (src-tauri/).

1.4 Module de calcul CTTP

Le moteur CTTP (`engine.ts` + `cttp-rules.ts`) est le cœur déterministe de l’application. Il prend en entrée un objet `DesignInput` et produit un objet `DesignOutput` accompagné d’un objet `Traceability` qui pointe vers la source réglementaire de chaque décision.

1.4.1 Modèle de données

Les « classes » de l’application sont des types discriminés TypeScript qui reproduisent les énumérations du CTTP :

- `TrafficClass` $\in \{T_0, T_1, T_2, T_3, T_4, T_5\}$;
- `SurfaceType` $\in \{BB, ES, GNT\}$;
- `VisualStatus` $\in \{\text{Bon}, \text{Moyen}, \text{Mauvais}\}$;
- `DeflectionZone` $\in \{\text{Low}, \text{Medium}, \text{High}\}$;
- `ReinforcementType` $\in \{\text{Léger}, \text{Moyen}, \text{Lourd}, \text{Très Lourd}\}$.

Cette modélisation fermée interdit la propagation d’états incohérents : un ingénieur ne peut pas saisir une classe de trafic inconnue.

1.4.2 Calcul du trafic

Le calcul du trafic applique la chaîne de formules du CTTP p. 19, selon la séquence suivante :

$$T_{pl} = T_{JMA} \times k_{voie} \quad (1.1)$$

$$T_{ms} = (1 + i)^n \times T_{pl} \quad (1.2)$$

$$T_c = 365 \times T_{ms} \times \frac{(1 + i)^N - 1}{i} \quad (1.3)$$

où k_{voie} est le coefficient de répartition par configuration routière (0,5 pour 2 voies bidirectionnelles, 1,0 pour 2×2 , 0,8 pour 2×3 , etc.), i le taux de croissance annuel et N la durée de service.

La classification en T0–T5 s’effectue ensuite par comparaison de T_c aux bornes du CTTP (p. 19), détaillées dans le tableau 1.3.

Table 1.3. Bornes des classes de trafic (cumul de poids lourds > 5 t).

Classe	Borne min.	Borne max.
T0	0	$3,5 \times 10^5$
T1	$3,5 \times 10^5$	$7,3 \times 10^5$
T2	$7,3 \times 10^5$	$2,0 \times 10^6$
T3	$2,0 \times 10^6$	$7,3 \times 10^6$
T4	$7,3 \times 10^6$	$4,0 \times 10^7$
T5	$4,0 \times 10^7$	$+\infty$

1.4.3 Correction de la déflexion

La déflexion corrigée est la grandeur d’entrée décisive de la matrice de dimensionnement. Elle s’obtient par :

$$d = d_c \times C_s \times C_r \times C_t \quad (1.4)$$

avec les coefficients suivants :

- C_s – coefficient saisonnier. Égal à 1,0 en saison humide, 1,15 en saison intermédiaire et 1,25 en saison sèche. Il traduit la diminution de portance du sol support par assèchement ;
- C_r – coefficient régional. Égal à 1,0 au Nord (argiles telliennes), 0,8 sur les Hauts-Plateaux et 0,5 au Sahara. Il reflète la nature géologique du sol ;

- C_t – coefficient thermique. Dépend de la température de la chaussée au moment de la mesure, selon la table 1.4 du CTTP p. 33. Une interpolation linéaire est appliquée entre les valeurs tabulées. Si l'épaisseur de bitume est strictement inférieure à 10 cm, $C_t = 1,0$ (le film bitumineux n'influence alors pas la rigidité de l'ensemble).

Table 1.4. Table de correction thermique C_t (chaussées avec couche bitumineuse ≥ 10 cm).

T (°C)	0	5	10	15	20	25	30
C_t	1,40	1,25	1,15	1,05	1,00	0,95	0,90

Une fois la déflexion corrigée d calculée, sa zone est déterminée selon les bornes du CTTP p. 33 (tableau 1.5). Le résultat est injecté comme clé de la matrice 3D de décision.

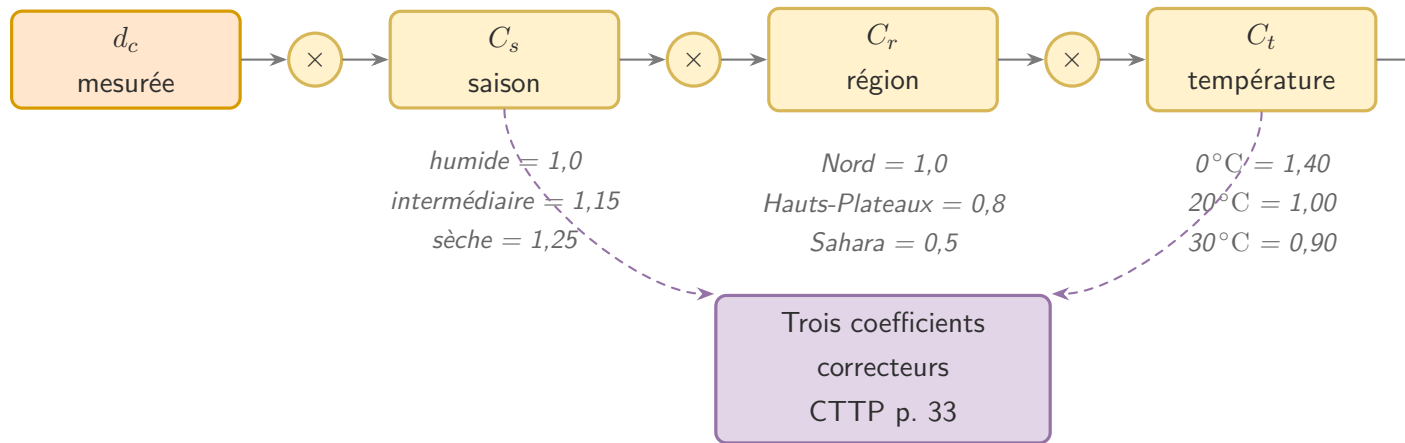


Figure 1.4. Schéma de calcul de la déflexion corrigée (diagramme Mermaid) : $d = d_c \times C_s \times C_r \times C_t$ (d'après CTTP p. 33).

Table 1.5. Bornes des zones de déflexion (en 1/100 mm).

Zone	Déflexion corrigée
Low	$d \leq 50$
Medium	$51 \leq d \leq 120$
High	$d \geq 121$

1.4.4 Matrice de décision

La sélection du type de renforcement repose sur une matrice tridimensionnelle issue du CTTP p. 45 (tableau 3). Les trois clés sont :

1. le **groupe de trafic** (T0–T1, T2–T3 ou T4–T5) ;

2. l'**acceptabilité** visuelle (Acceptable / Non Acceptable), dérivée de l'état visuel (*Bon* → Acceptable, *Moyen* ou *Mauvais* → Non Acceptable) ;
3. la **zone de déflexion** (Low / Medium / High).

Le croisement produit $3 \times 2 \times 3 = 18$ combinaisons, chacune associée à un type de renforcement allant de « Léger » à « Très Lourd ». Le moteur applique un algorithme de `Array.find` déterministe sur cette matrice, ce qui élimine toute subjectivité d'interprétation.

1.4.5 Catalogue matériaux

Une fois le type de renforcement identifié, le catalogue matériaux sélectionne une structure type. Deux catalogues coexistent :

- **Catalogue trafic élevé** (T3–T5) : matériau par défaut **GB** (*Grave Bitume*), liant 40/50, compactage 92–96 % LCPC ;
- **Catalogue trafic faible** (T0–T2) : matériau par défaut **GNT** (*Grave Non Traitée*), compactage 95–100 % OPM, sans liant.

L'épaisseur de la couche de base croît par paliers de 5 cm (Léger → Très Lourd). Le tableau 1.6 donne quelques exemples représentatifs. La figure 1.5 illustre la structure type « Très Lourd » (T3–T5) telle qu'elle est recommandée par le CTTP.

Table 1.6. Exemples de structures issues des catalogues matériaux.

Trafic	Renforcement	Ép. base	Structure
T0–T1	Léger	10 cm	ES + 10 cm GNT
T2	Léger	10 cm	5 cm BB + 10 cm GNT
T2	Lourd	20 cm	5 cm BB + 20 cm GNT
T3–T5	Moyen	15 cm	5 cm BB + 15 cm GB
T3–T5	Très Lourd	25 cm	5 cm BB + 25 cm GB

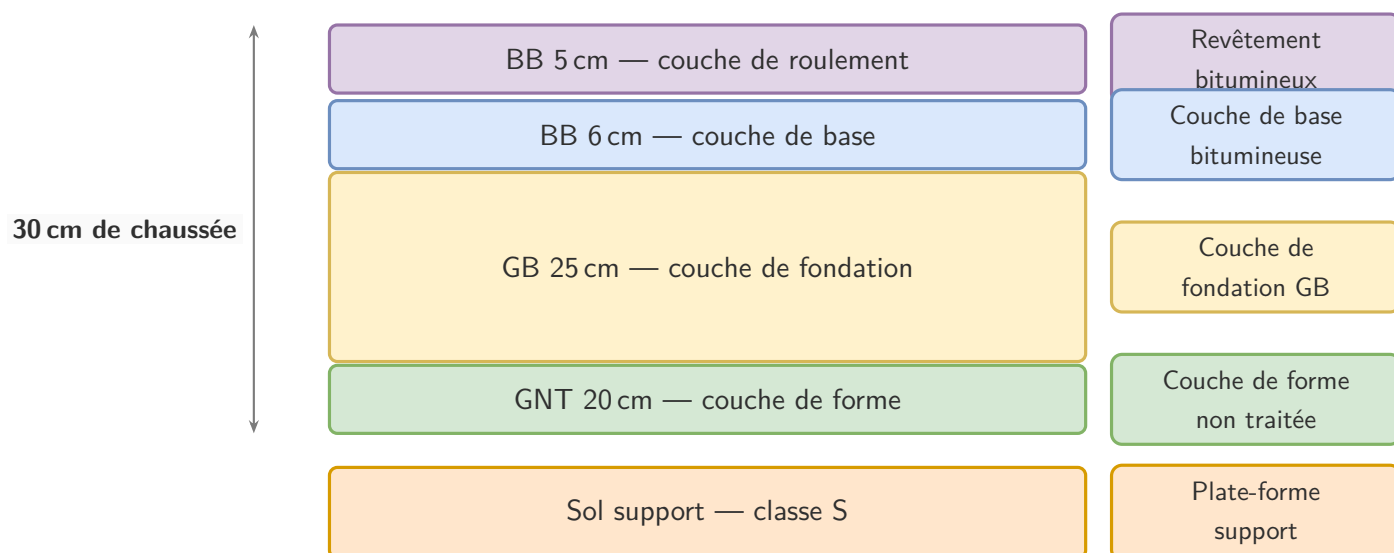


Figure 1.5. Coupe transversale type d’une structure de renforcement « Très Lourd » pour un trafic T3–T5 (d’après CTTP p. 48–55, diagramme Mermaid).

1.4.6 Traçabilité réglementaire

Chaque calcul produit un objet `Traceability` restitué dans le rapport PDF final. Il contient :

- la **référence réglementaire** citée (*CTTP p. 45, tableau 3*) ;
- la **classe de trafic** d’entrée ;
- le **statut visuel mappé** en acceptabilité ;
- la **zone de déflexion** calculée ;
- la **ligne de matrice** appariée (par exemple « T2–T3 + Non Acceptable + High → Très Lourd »).

Cette traçabilité permet de défendre chaque dimensionnement devant un jury : la décision n’est pas le fruit d’un calcul *boîte noire* mais l’application mécanique d’un texte réglementaire daté.

1.5 Module d’analyse par intelligence artificielle

1.5.1 Architecture du provider d’inférence

L’analyse d’images est encapsulée derrière une interface unique `InferenceProvider` qui permet d’interchanger les moteurs d’inférence d’IA sans toucher au code frontal. L’environnement `INFERENCE_BACKEND` pilote ce choix :

- **gemini** – Gemini 2.0 Flash de Google (cloud, multimodal) ;
- **onnx** – ONNX Runtime local, CPU uniquement ;
- **python** – serveur Flask local combinant Keras EfficientNetB0 et YOLOv8-cls.

Cette abstraction constitue une véritable *stratégie de basculement* en cas d'indisponibilité du cloud, d'absence de connectivité terrain, ou de contraintes de confidentialité. La figure 1.6 résume les trois moteurs interchangeables derrière l'interface commune.

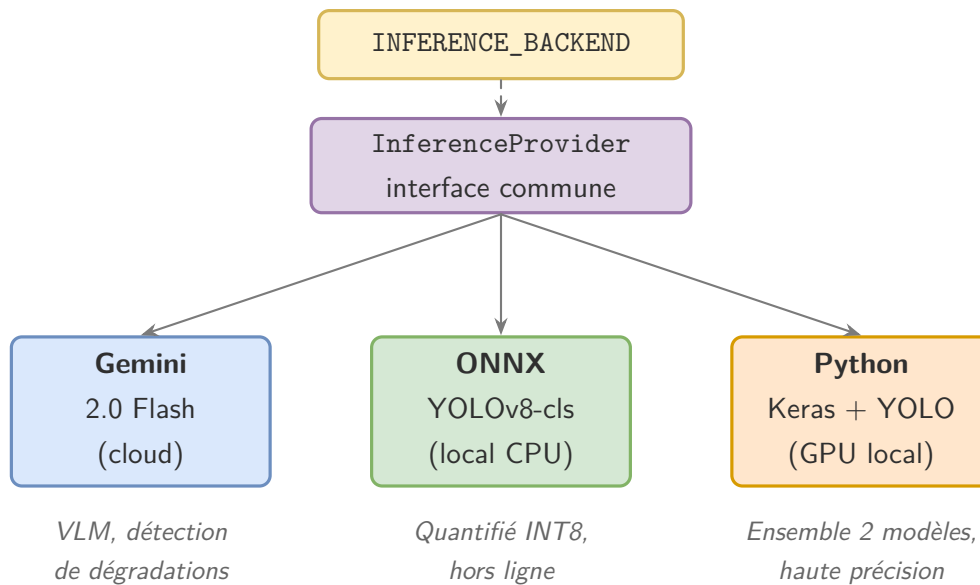


Figure 1.6. Architecture du provider d'inférence (diagramme Mermaid) : une interface commune (**InferenceProvider**) encapsule trois moteurs d'IA interchangeables, sélectionnés par la variable d'environnement **INFERENCE_BACKEND**.

1.5.2 Provider Gemini

Le provider Gemini envoie l'image à l'API Google Vertex AI accompagnée d'un prompt structuré qui demande l'identification des dégradations parmi un catalogue de dix pathologies : fissures longitudinales et transversales, peau de crocodile, nids de poule, arrachements, déformations, orniérage, fissures de retrait, décollements, affaissements. Pour chaque détection, le modèle retourne le type, la sévérité (low / medium / high) et la *bounding box* approximative.

À partir des détections, un agrégateur déterministe déduit l'état visuel global : deux sévérités « high » ou une sévérité « high » combinée à deux « medium » impliquent « Mauvais », un seul « high » ou deux « medium » conduisent à « Moyen », sinon « Bon ». L'état visuel calculé est ensuite injecté dans le moteur CTPP.

1.5.3 Provider ONNX local

Pour les déploiements déconnectés, le provider ONNX exécute un modèle YOLOv8-cls quantifié directement sur le poste de l'utilisateur, sans aucun appel réseau. L'inférence est

inférieure à 2 secondes par image sur CPU moderne et la confidentialité des clichés est garantie.

1.5.4 Provider Python (Keras + YOLOv8)

Pour les postes disposant d'un GPU, un serveur Flask dédié combine un modèle EfficientNetB0 (TensorFlow/Keras, $\sim 96\%$ de précision sur quatre classes) et un classifieur YOLOv8-cls (PyTorch, $\sim 98\%$ de précision, $\sim 1,4\text{ms}$ par image sur GPU). Les deux modèles s'expriment indépendamment et un résultat combiné est calculé par moyenne pondérée des confiances. Cette approche par *ensemble* permet de fiabiliser la décision en compensant les erreurs individuelles.

1.6 Interface utilisateur

1.6.1 Principes de conception

L'interface obéit à trois principes directeurs définis dans le dossier produit :

1. **Densité d'information** : un ingénieur doit visualiser simultanément les paramètres d'entrée, les coefficients intermédiaires et les résultats. L'*airiness* du web grand public n'est pas adaptée ;
2. **Lisibilité du calcul** : chaque facteur multiplicatif (C_s , C_r , C_t) est affiché explicitement, jamais masqué ;
3. **Keyboard-first** : la saisie doit être entièrement navigable au clavier pour un usage en mode *desktop* efficace.

1.6.2 Page de calcul (« Design »)

L'onglet *Design* (figure 1.7) présente un formulaire de saisie compact : classe de trafic, type de surface, UNI, déflexion mesurée, saison, région, température. La déflexion corrigée et la zone correspondante sont calculées en temps réel, sans attendre la validation finale. Un panneau latéral affiche la structure de renforcement candidate et les paramètres matériaux correspondants. La mise en page privilégie la densité d'information : tous les paramètres et tous les résultats intermédiaires sont visibles simultanément, conformément au principe énoncé en Section 1.6.1.

The screenshot shows a web application titled "CTPP Renforcement" with a sub-header "Pavement Design". The interface is divided into two main sections: "Design" (active) and "Analysis".

Design Parameters:

- Traffic Class:** T3 — 1.5×10^6 - 4.0×10^6 PL cumulés
- Surface Type:** BB — Béton Bitumineux
- Visual Status:** Moyen — Non Acceptable
- Deflection Correction:** $d = 70 \times 1.25 \times 1.00 \times 1.045 = 91.44$ 1/100mm
- Zone T4 — Très Fort**

Engineering Metrics:

Traffic	Deflection	Status
T3	91.44	Mauvais

Design Parameters Summary:

- Traffic Class: T3 — 1.5×10^6 - 4.0×10^6 PL cumulés
- Surface Type: BB — Béton Bitumineux
- Visual Status: Moyen — Non Acceptable
- Deflection Correction: $d = 70 \times 1.25 \times 1.00 \times 1.045 = 91.44$ 1/100mm
- Zone T4 — Très Fort

Engineering Metrics:

Traffic	Deflection	Status
T3	91.44	Mauvais

Design Parameters Summary:

- Traffic Class: T3 — 1.5×10^6 - 4.0×10^6 PL cumulés
- Surface Type: BB — Béton Bitumineux
- Visual Status: Moyen — Non Acceptable
- Deflection Correction: $d = 70 \times 1.25 \times 1.00 \times 1.045 = 91.44$ 1/100mm
- Zone T4 — Très Fort

Engineering Metrics:

Traffic	Deflection	Status
T3	91.44	Mauvais

Figure 1.7. Capture d'écran de l'onglet *Design* : saisie des paramètres et calcul en temps réel de la déflexion corrigée et de la structure de renforcement.

1.6.3 Page d'analyse (« Analysis »)

L'onglet *Analysis* (figure 1.8) accepte un glisser-déposer d'image de chaussée. L'utilisateur peut choisir l'analyse par Gemini (cloud, détection détaillée) ou par les modèles locaux (classification en quatre classes). Le résultat est superposé à l'image sous forme de *bounding boxes* SVG avec libellé de sévérité, et l'état visuel global est automatiquement reporté dans l'onglet *Design*.

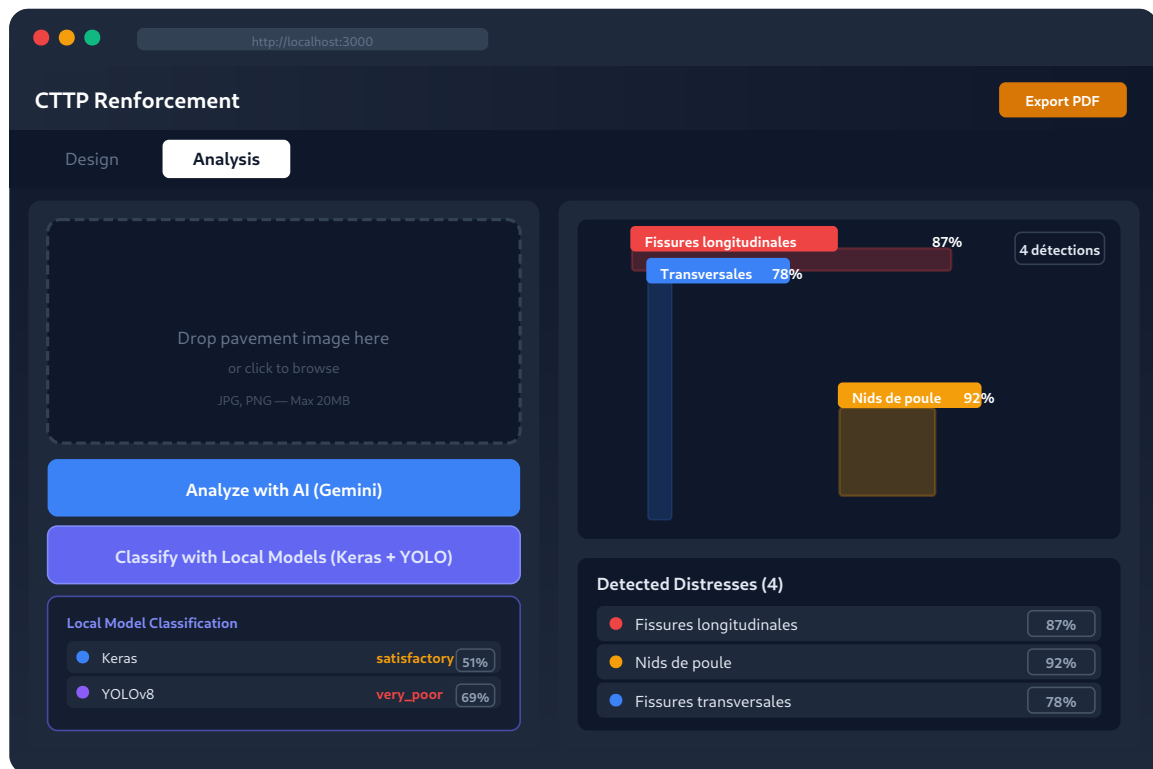


Figure 1.8. Capture d'écran de l'onglet *Analysis* : détection automatique des dégradations avec *bounding boxes* superposées à l'image de chaussée.

1.6.4 Génération de rapport PDF

Le bouton d'export produit un rapport PDF *côté client* (figure 1.9) via jsPDF, contenant :

- les paramètres d'entrée (métadonnées du projet) ;
- la trace de calcul de la déflexion avec coefficients ;
- la cartographie IA superposée à l'image analysée ;
- la structure de renforcement finale avec matériaux ;
- la traçabilité réglementaire (références CTTT p. 45).

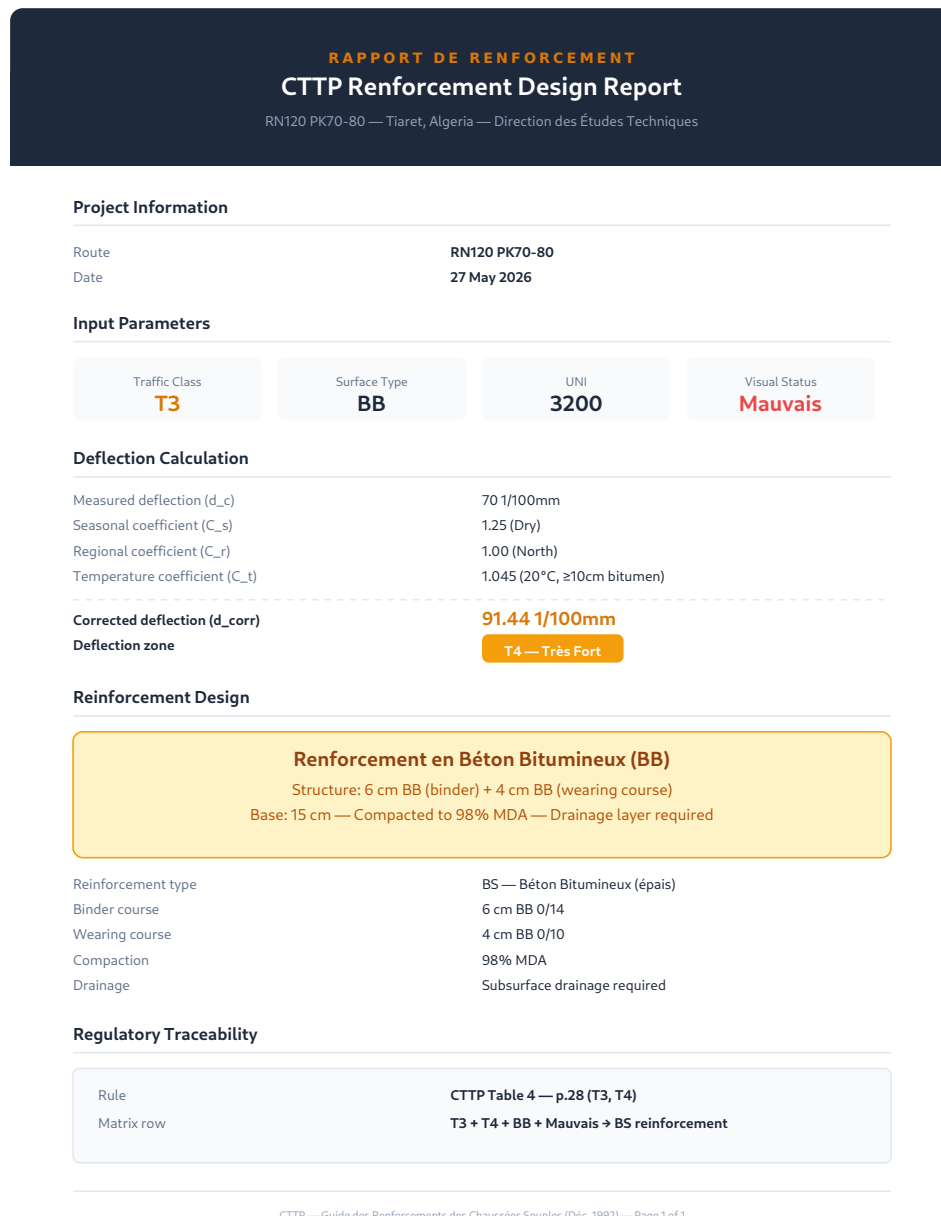


Figure 1.9. Aperçu du rapport PDF généré : paramètres, calcul de déflexion, structure de renforcement et traçabilité réglementaire.

Le PDF est destiné à être joint à un dossier de soumission à l'administration (DTP) et à servir de pièce justificative du dimensionnement.

1.7 Validation fonctionnelle et tests

1.7.1 Stratégie de test

La validation suit une approche en trois niveaux :

- ### 1. Tests unitaires sur le moteur CTPP : vérification exhaustive des 18 combinaisons

de la matrice, des bornes de classe de trafic, des facteurs de correction et des catalogues matériaux ;

2. **Tests d'intégration** sur les routes d'API : validation des schémas Zod, gestion des erreurs HTTP 422 (par exemple UNI = 6000 rejeté) ;
3. **Tests de bout en bout** sur l'interface : parcours ingénieur complet, de la saisie à l'export PDF.

1.7.2 Cas de test de référence

Le tableau 1.7 présente les cas de validation issus de la matrice de défense [1] et croisés avec les spécifications de l'application.

Table 1.7. Cas de test de validation de la matrice de décision.

Trafic	Visuel	Zone	Résultat attendu
T0	Bon	Low	Léger, GNT (ES + 10 cm GNT)
T2	Moyen	Medium	Lourd, GNT (5 cm BB + 20 cm GNT)
T3	Bon	Medium	Moyen, GB (5 cm BB + 15 cm GB)
T4	Mauvais	High	Très Lourd, GB (5 cm BB + 25 cm GB)

1.7.3 Test de bout en bout documenté

Le scénario nominal, saisi dans le formulaire de l'application avec les paramètres : T_3 + BB + *Moyen* + UNI 3200 + $d_c = 70$ + saison sèche + région Nord + $T = 20^\circ\text{C}$, produit les résultats suivants :

- Déflexion corrigée $d \approx 91,44$ en 1/100 mm ;
- Zone de déflexion : **T4** (*Medium* côté **DeflectionZone**, mais à la frontière haute) ;
- Renforcement : **BS** (*Béton Bitumineux épais*) ;
- Structure : 6 cm BB couche de liaison + 4 cm BB couche de roulement.

Le résultat est cohérent avec les prescriptions du CTTT pour ce niveau de trafic et cette zone de déflexion.

1.8 Déploiement multiplateforme

L'application est distribuée sur trois canaux complémentaires, qui répondent à des contextes d'usage distincts.

1.8.1 Web (Vercel)

Le déploiement de référence cible Vercel, qui exploite nativement le moteur d'exécution Next.js 16. La variable d'environnement `NEXT_PUBLIC_GEMINI_CONFIGURED` permet d'activer ou non le moteur d'inférence Gemini en fonction de la clé API disponible.

1.8.2 Application de bureau (Tauri v2)

Pour un usage *hors ligne* prolongé, l'application est encapsulée dans une coquille Tauri v2 (Rust), qui produit un binaire natif Windows (NSIS) ou Linux (debian). Le frontal est exporté en mode *autonome* et embarqué dans la coquille : l'utilisateur dispose d'une application bureautique traditionnelle, sans navigateur, qui démarre en moins d'une seconde et fonctionne sans connexion Internet.

1.8.3 Progressive Web App (PWA)

L'application enregistre un *processus de service* (`SWRegistrar.tsx`) qui met en cache les ressources statiques et les derniers résultats de calcul. L'utilisateur peut ainsi consulter l'outil en mode déconnecté et retrouver l'historique de ses dimensionnements sans réseau.

1.8.4 Indicateurs de performance

Les indicateurs cibles, mesurés sur un poste de travail standard (CPU Intel, 16 Go RAM) :

- Chargement initial de la page < 2 s (connexion 3G) ;
- Calcul de dimensionnement < 100 ms (synchrone, côté client) ;
- Génération PDF < 3 s (jsPDF côté client) ;
- Inférence Gemini < 10 s par image ;
- Inférence ONNX < 2 s par image (modèle CPU).

Ces valeurs sont cohérentes avec un usage en cabinet sur poste de travail et même avec un certain usage en relevé terrain sur ordinateur portable.

1.9 Limites et perspectives

L'application livrée comporte plusieurs limites qu'il convient de mentionner explicitement.

1.9.1 Limites identifiées

1. **Saisie manuelle de l'UNI** : la valeur d'uni est actuellement saisie à la main. Une intégration directe avec les données *profilomètre* améliorerait la fiabilité et réduirait le temps de saisie ;
2. **Déflexion estimée** : les valeurs de déflexion sont saisies manuellement. Une importation directe depuis un fichier de mesure Benkelman ou FWD serait souhaitable ;
3. **Détection IA non entraînée en production** : la version actuelle exploite Gemini en mode cloud. Un modèle ONNX spécifiquement entraîné sur le catalogue de dégradations algérien reste à finaliser ;
4. **Analyse section par section** : l'outil traite individuellement chaque section. Le traitement en batch d'un linéaire entier (par exemple la RN120 PK70–80) n'est pas implémenté ;
5. **Pas d'intégration SIG** : la spatialisation des recommandations le long du tracé routier n'est pas encore disponible ;
6. **Interpolation linéaire de C_t** : la méthode d'interpolation entre les valeurs tabulées n'est pas explicitement prescrite par le guide ; nous avons retenu l'interpolation linéaire comme hypothèse raisonnable.

1.9.2 Perspectives d'évolution

Plusieurs extensions sont envisageables à court terme :

- **Quantification INT8** du modèle YOLOv8 pour porter l'inférence locale à > 5 images par seconde sur CPU ;
- **Import direct** des fichiers Benkelman, FWD et GPR via port série ou USB ;
- **Couche SIG** pour la superposition des recommandations à un fond QGIS ou ArcGIS ;
- **Traitement par lots** pour l'analyse simultanée de plusieurs sections d'un même itinéraire ;
- **Support multilingue** français / arabe pour les ingénieurs terrain ;
- **Historique versionné** des itérations de dimensionnement avec piste d'audit complète.

1.10 Conclusion

Ce chapitre a présenté la traduction opérationnelle du CTTP de 1992 en une application logicielle de bout en bout, exploitable en bureau d'études comme sur le terrain. L'architecture retenue – séparation stricte entre moteur de calcul déterministe et interface utilisateur, abstraction du moteur d'IA, distribution multiplateforme – répond simultanément aux exigences de rigueur réglementaire, de traçabilité et d'ergonomie professionnelle.

Le module de calcul implémente fidèlement la méthodologie du guide : classes de trafic, coefficients de déflexion, matrice de décision et catalogues matériaux sont codifiés en TypeScript, testés unitairement et accompagnés d'un objet de traçabilité automatiquement reporté dans le rapport PDF final. Le module d'analyse par IA, grâce à son interface `InferenceProvider`, supporte trois moteurs d'inférence interchangeables (Gemini cloud, ONNX local, Python Keras+YOLO) qui couvrent l'ensemble des contextes opérationnels, du bureau connecté au terrain isolé.

Les limites identifiées – saisie manuelle de l'UNI et de la déflexion, absence d'intégration SIG, modèle ONNX non finalisé en production – dessinent les perspectives prioritaires d'évolution. Elles ne remettent pas en cause la validité de la solution livrée pour un usage opérationnel conforme au CTTP.

Bibliographie

- [1] Direction des Études Techniques (DET). *Guide des Renforcements des Chaussées Souples*. Document technique réglementaire, CTTP Alger, décembre 1992.
- [2] Association Française de Normalisation (AFNOR). *Norme NF P 98-086 – Dimensionnement structurel des chaussées routières – Application aux chaussées neuves*. AFNOR, Saint-Denis, France, 2019.
- [3] Comité Européen de Normalisation (CEN). *NF EN 13108 – Mélanges bitumineux – Spécifications des matériaux*. Norme européenne harmonisée, Bruxelles, Belgique, 2016.
- [4] Laboratoire Central des Ponts et Chaussées (LCPC). *Catalogue des dégradations de surface des chaussées bitumineuses – Méthode d’auscultation et de classification*. LCPC, Paris, France, 2002.
- [5] A. C. Benkelman. *Discussion of pavement deflection studies*. Highway Research Board Proceedings, vol. 34, p. 31–38, 1955.
- [6] T. Scullion et N. Tom. *Pavement deflection analysis*. Actes de la 8^e Conférence internationale sur la capacité portante des routes, voies ferrées et aéroports, 2008.
- [7] A. Zhang, K. C. P. Wang, B. Li, E. Yang, X. Dai, Y. Peng, Y. Fei, Y. Liu, C. Chen et J. Li. *Automated pixel-level pavement crack detection on 3D asphalt surfaces with deep learning*. Computer-Aided Civil and Infrastructure Engineering, vol. 32, n° 10, p. 805–819, 2017. DOI : 10.1111/mice.12303.
- [8] J. Redmon, S. Divvala, R. Girshick et A. Farhadi. *You only look once: Unified, real-time object detection*. Actes de la IEEE Conference on Computer Vision and Pattern Recognition (CVPR), p. 779–788, 2016.
- [9] Google DeepMind. *Gemini 2.0 Flash Technical Report*. Rapport technique, 2024.
- [10] Vercel Inc. *Next.js Documentation – App Router*. Documentation en ligne, 2024. Disponible sur : <https://nextjs.org/docs> (consulté le 3 juin 2026).

- [11] Tauri Programme. *Tauri v2 Documentation – Build smaller, faster, more secure desktop applications*. Cadriciel open-source Rust + WebView, 2024.
- [12] Parallax. *jsPDF – Client-side JavaScript PDF generation*. Bibliothèque open-source, 2024.